moOngy.
Minds on the move

React Avançado · Módulos 1–2

Sessão 2

React 19 na prática

qui 5 nov 2026 · 19h–21h

João Gonçalves · joaogoncalves.net

Recap da Sessão 1

Parámos no FlightCard — tag sessao-01

Setup

- Vite + TypeScript (`strict`)
- `json-server@0.17` + `run-p`
- `features/` e `ui/`
- `import type` · sem `enum`

Componentes tipados

- Props tipadas na desestruturação
- `ReactNode` para children e slots
- `ComponentProps<'button'>` + `...rest`

FlightCard

- **Não** guarda estado
- `selected` para baixo, `onSelectedChange` para cima
- Derivados calculam-se no render

Por resolver: o card avisa que mudou… mas **quem guarda** os voos selecionados? → `FlightList` + cesto, já a seguir.

Primeiro: o ambiente de todos

5 minutos agora poupam a sessão inteira

```
terminal
node -v                  # 24 LTS: 24.15 ou superior

git clone https://github.com/JoaoGoncalves/skyline.git
cd skyline
git checkout sessao-01   # quem já clonou: git pull
npm install
npm start

# → json-server  http://localhost:3000/flights
# → Vite         http://localhost:5173
```

Problemas comuns

- Node < 24.15 → instalar o **24 LTS** mais recente
- `npm install` bloqueado → proxy da empresa
- Porta 3000 ocupada → `--port 3001` no script `api`
- TS diferente? Conta o **do projeto**: `npm ls typescript`
- `git pull` trouxe `package.json` novo → `npm install`

Quem ainda não tiver o ambiente a funcionar: acompanha no ecrã partilhado e resolvemos no fim — a turma não pára.

Agenda de hoje

00	**Revisão: componentes tipados**	normais vs que estendem nativos · React 18 → 19	≈ 15 min
01	**Fechar a lista de voos**	FlightList · keys · o cesto · Badge	≈ 16 min
02	**React 19: o que mudou**	ref como prop · useId · <title> · FlightSearch	≈ 25 min
→	**Na Sessão 3**	Revisão de Hooks · rendering e reconciliation	S3

Revisão

Componentes tipados

O que ficou rápido demais na Sessão 1 — com calma, e do React 18 para o 19

Dois tipos de componente tipado

Tudo o que fizemos na Sessão 1 cabe numa destas duas colunas

	Componente "normal"	Estende um elemento nativo
No Skyline	Panel · FlightCard	Button · ExternalLink · Badge
Props	um type só com as nossas props	ComponentProps<'x'> & as nossas
Opcionais e defaults	prop?: tipo · default na desestruturação	igual (ex.: type = 'button')
children / slots	declarados: ReactNode	já vêm do elemento nativo
O resto	—	...rest segue para o elemento
No React 18	propTypes · defaultProps · React.FC	forwardRef · XxxHTMLAttributes<…>

A pergunta a fazer sempre: **o meu componente é um elemento nativo "melhorado"?** Se sim → `ComponentProps<'x'> + ...rest`.



Da teoria ao código

Agora, vamos escrever o código

- Uma pasta **temporária** `src/revisao/` — apaga-se no fim deste bloco
- `Greeting`: componente normal (obrigatória, opcional, callback, children)
- `ExternalLink`: estende o `<a>` · e vemos os erros que o TypeScript apanha

Ficheiros: src/revisao/Greeting.tsx · ExternalLink.tsx · RevisaoPage.tsx · App.tsx (temporário)

Componente normal: do React 18 ao 19

Greeting — props nossas, tipadas na desestruturação

antes

```
// React 16/18 · JavaScript
function Greeting({ name, excited, children }) { … }
Greeting.propTypes = {
  name: PropTypes.string.isRequired,
  excited: PropTypes.bool };
Greeting.defaultProps = { excited: false };

// React 18 · TypeScript
const Greeting: React.FC<Props> = …
// FC dava children "de graça"
```

React 19 · src/revisao/Greeting.tsx

```
type GreetingProps = {
  name: string;                    // obrigatória
  excited?: boolean;               // opcional
  onGreet?: (name: string) => void; // callback
  children?: ReactNode;
};

export function Greeting(
  { name, excited = false, onGreet, children }: GreetingProps
) {
  return <div className="card">…</div>;
}
```

propTypes e defaultProps são **ignorados** no React 19 → validação passa para o TypeScript (compilação), defaults para a desestruturação.

Estender um elemento nativo: do React 18 ao 19

ExternalLink — tudo o que um <a> aceita, mais as nossas regras

antes

```tsx
// React 18 · TypeScript
type Props =
  React.AnchorHTMLAttributes<HTMLAnchorElement>;

const ExternalLink =
  forwardRef<HTMLAnchorElement, Props>(
    ({ children, ...rest }, ref) => (
      <a ref={ref} {...rest}>{children} ↗</a>
    ));
```

```tsx
React 19 · src/revisao/ExternalLink.tsx
type ExternalLinkProps = ComponentProps<'a'> & {
  href: string;   // no <a> é opcional: aqui obrigamos
};

export function ExternalLink(
  { children, ...rest }: ExternalLinkProps
) {
  return (
    <a target="_blank" rel="noreferrer" {...rest}>
      {children} ↗
    </a>
  );
}
```

`ComponentProps<'a'>` já traz a `ref` → sem `forwardRef`. O `...rest` leva `title`, `className`, `aria-*`… sem as declararmos.

Bloco 01

Fechar a lista de voos

Do FlightCard à FlightList — onde parámos na Sessão 1



Da teoria ao código

Agora, vamos escrever o código

- FlightList: um FlightCard por voo, com `key`
- O cesto: `Record<number, boolean>` — quem guarda a seleção
- Um App temporário com os 10 voos (substitui o da revisão)

Ficheiros: features/flights/FlightList.tsx · App.tsx

FlightList: uma lista com keys e o cesto

```tsx
type FlightListProps = {
  flights: Flight[];
  basket: Record<number, boolean>;
  onToggle: (flightId: number, selected: boolean) => void;
};

export function FlightList({ flights, basket, onToggle }: FlightListProps) {
  if (flights.length === 0) {
    return <p className="empty">No flights found.</p>;
  }
  return (
    <div className="grid">
      {flights.map((flight) => (
        <FlightCard
          key={flight.id}
          flight={flight}
          selected={basket[flight.id] ?? false}
          onSelectedChange={(selected) => onToggle(flight.id, selected)}
        />
      ))}
    </div>
  );
}
```

A reter

- `key` estável e única — **nunca** o índice numa lista que muda
- Early return para o caso vazio
- O callback fecha sobre `flight.id`
- Porque é que as keys importam: Bloco 04

A vossa vez: um Badge para o FlightCard

3 minutos — depois escrevemos juntos

O que o Badge tem de fazer

- Ficheiro: `src/ui/Badge.tsx` — estende o `<span>`
- Aceita **tudo** o que um `<span>` aceita (`title`, `onClick`, `aria-*`…)
- Prop `variant`: só `'info'` ou `'warn'` — por omissão `'info'`
- CSS: `'warn'` → classe `badge` · `'info'` → classe `chip`
- E não perde o `className` que vier de fora

FlightCard.tsx

```tsx
// header do FlightCard
{flight.delayed ? (
  <Badge variant="warn">
    +{flight.delay} min
  </Badge>
) : (
  <Badge>on time</Badge>
)}

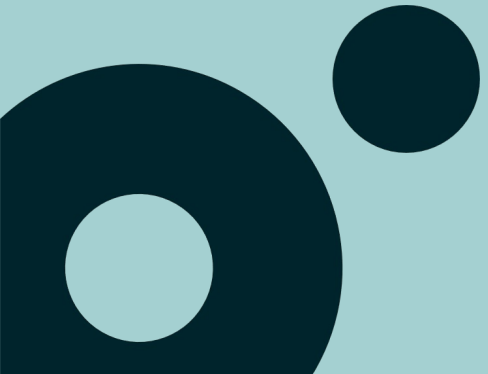
<Badge variant="danger">  // erro
```

Pista: é o mesmo padrão do `ExternalLink` da revisão — um `type` com `ComponentProps<'x'> & { … }` e uma função com `...rest`.

Bloco 02

React 19: o que mudou

O que entrou, o que saiu — e o que fica para as próximas sessões



O que o React 19 trouxe — e quando o vemos

Novidade	Para quê	Sessão
ref como prop	adeus forwardRef	**hoje**
<title> · <meta> · <link>	metadata a partir de qualquer componente	**hoje**
<Context value>	o Context é o provider	**hoje · S10**
use() · ref cleanup · useEffectEvent	promises/context · refs · effects	S4
React Compiler	automatic memoization	S3
Actions · useActionState · useOptimistic	formulários e mutações	S7 – S8
<Activity> · <ViewTransition>	concurrent UX	S11
Server Components · Server Actions	o lado do servidor	S11

O que saiu — ou deixou de ser preciso

Antes	Agora (React 19)
forwardRef((props, ref) => …)	function X({ ref, …props })
Comp.propTypes = { … }	TypeScript (propTypes são ignorados)
Comp.defaultProps = { … } (functional components)	default no parâmetro: { size = 'md' }
<ThemeContext.Provider value={…}>	<ThemeContext value={…}>
ReactDOM.render(<App />, el)	createRoot(el).render(<App />)
ref="input" · contextTypes	removidos (eram de class components)

Código antigo? Codemods oficiais (`npx codemod react/19/…`): `migration-recipe` (render, string refs, propTypes) · `remove-forward-ref` · `remove-context-provider`.

ref como prop · Context como provider

```tsx
// antes (React 18)
const SearchInput = forwardRef<HTMLInputElement, Props>(
  (props, ref) => <input ref={ref} {...props} />
);

// agora (React 19)
function SearchInput({ ref, ...props }: Props) {
  return <input ref={ref} {...props} />;
}
```

```tsx
const ThemeContext = createContext<Theme>('light');

// provider: o próprio Context
<ThemeContext value="dark">
  {children}
</ThemeContext>

// consumir
const theme = use(ThemeContext);
```

Hoje usamos a ref no Skyline. O Context fica para a Sessão 10 — e o `use()` para a Sessão 4.



Da teoria ao código

Agora, vamos escrever o código

- SearchInput: ref como prop, useId para ligar label e input
- FlightSearch: o filtro, a pesquisa, o cesto e um <title> dinâmico
- Ligar tudo no App

Ficheiros: features/flights/SearchInput.tsx · FlightSearch.tsx · App.tsx

SearchInput: ref como prop e useId

```tsx
import { useId, type ComponentProps } from 'react';

type SearchInputProps = ComponentProps<'input'> & { label: string };

export function SearchInput({ label, ref, ...rest }: SearchInputProps) {
  const id = useId();
  return (
    <div className="field">
      <label htmlFor={id}>{label}</label>
      <input id={id} ref={ref} {...rest} />
    </div>
  );
}

// FlightSearch.tsx
const fromRef = useRef<HTMLInputElement>(null);
<SearchInput ref={fromRef} label="From" name="from" … />
fromRef.current?.focus();   // no clear()
```

A reter

- `ComponentProps<'input'>` já inclui `ref`
- Sem `forwardRef`
- `useId`: ids únicos e estáveis para `label`/`input`
- `?.focus()`: a ref pode ser `null`

FlightSearch (1): state, derivados e handlers

```tsx
const [filter, setFilter] = useState<Filter>({ from: 'Lisbon', to: 'Porto' });
const [flights, setFlights] = useState<Flight[]>([]);
const [basket, setBasket] = useState<Record<number, boolean>>({ 5: true, 7: true });
const fromRef = useRef<HTMLInputElement>(null);

// derivados: calculados em cada render
const canSearch = filter.from !== '' && filter.to !== '';
const selectedCount = Object.values(basket).filter(Boolean).length;

function handleChange(event: ChangeEvent<HTMLInputElement>) {
  const { name, value } = event.target;
  setFilter((f) => ({ ...f, [name]: value }));
}
function handleSubmit(event: SubmitEvent<HTMLFormElement>) {
  event.preventDefault();
  setFlights(sampleFlights.filter((f) =>
    f.from === filter.from && f.to === filter.to));
}
function updateBasket(flightId: number, selected: boolean) {
  setBasket((b) => ({ ...b, [flightId]: selected }));
}
```

A reter

- Nova referência: `{ ...f }`
- Forma **funcional** do setter
- Derivados: **sem** state nem effect
- `SubmitEvent`, não `FormEvent`

FlightSearch (2): o JSX e um <title> dinâmico

```
return (
  <>
    <title>{`Skyline · ${filter.from || '…'} → ${filter.to || '…'}`}</title>

    <Panel title="Flight Search"
           actions={<span className="chip">{selectedCount} selected</span>}>
      <form className="search" onSubmit={handleSubmit}>
        <SearchInput ref={fromRef} label="From" name="from"
                     value={filter.from} onChange={handleChange} />
        <SearchInput label="To" name="to"
                     value={filter.to} onChange={handleChange} />
        <Button type="submit" disabled={!canSearch}>Search</Button>
        <Button variant="ghost" onClick={clear}>Clear</Button>
      </form>
      <FlightList flights={flights} basket={basket} onToggle={updateBasket} />
    </Panel>
  </>
);
```

A reter

- `<title>` sobe para o `<head>`
- Fragment `<>` para devolver vários nós
- `disabled` a partir de um derivado
- `clear()` → `fromRef.current?.focus()`

Resumo: React 19 na prática

Skyline

- `FlightList`: keys e o cesto
- `Badge`: estende o `<span>`
- `SearchInput`: `ref` como prop + `useId`
- `FlightSearch`: `<title>`, derivados, setters funcionais

React 19

- Adeus `forwardRef`
- `<title>` / `<meta>` em qualquer componente
- `<Context value>` · `use()`
- Saíram: `propTypes`, `defaultProps`

Componentes tipados

- Normal: um type só com as nossas props
- Estende nativo: `ComponentProps<'x'>` + `...rest`
- 18 → 19: sem `forwardRef`, `propTypes`, `defaultProps`

Na Sessão 3: revisão de Hooks e **porque é que um componente volta a renderizar** — escrever no From faz renderizar os 10 FlightCard. Porquê?

Exercício de revisão

Para casa · no Skyline, a partir da tag sessao-02 · corrigimos no início da Sessão 3

Obrigatório · ≈ 20 min

- Checkbox "Só voos sem atraso" (`useId` + `htmlFor`)
- Lista **derivada** — sem state novo nem effect
- O Clear desliga a checkbox

Extra

- Campo "Preço máx. (€)" reutilizando o `SearchInput`
- `type`, `min`, `placeholder` passam pelo `...rest`
- Mostrar "3 de 5 voos"

Bónus

- Ordenar por preço com `<select>`
- `ChangeEvent<HTMLSelectElement>`
- Ordenar sem mutar o state: `toSorted`

Enunciado no repo: `curso/exercicios/sessao-02.md` · antes de entregar: `npx oxlint` e `npm run build` sem erros

moOngy.
Minds on the move

Obrigado · até terça, 10 nov

edu@joaogoncalves.net · WhatsApp +351 910 346 437

`github.com/JoaoGoncalves/skyline · tag sessao-02`